

Last updated Nov 17, 2025

Worms

2ª fase do projeto de Laboratórios de Informática / Programação I
2025/26

Introdução

Neste enunciado apresentam-se as tarefas referentes à segunda fase do projeto das unidades curriculares de Laboratórios de Informática I e Laboratórios de Programação I.

Tarefas

Tarefa 4 - Implementar um *bot*

O objetivo desta tarefa é implementar um *bot*, na forma de uma função que determina uma tática para jogar automaticamente o jogo. Vamos considerar um cenário em que:

- Uma tática consiste numa sequência de exatamente 100 jogadas.
- O *bot* controla todas as minhocas.
- Para cada jogada efetuada (de acordo com a Tarefa 2), o jogo avança um instante de tempo (de acordo com a Tarefa 3).
- O objetivo da tática é obter pontuação máxima, em que maior pontuação está associada uma maior destruição causada no jogo. A pontuação é cumulativa e calculada da seguinte forma:
 - X pontos cada vez que uma minhoca perca X de vida ou tenha vida X e morra.
 - 10 pontos cada vez que um terreno do tipo Terra seja destruído.

A avaliação desta tarefa terá em conta a pontuação obtida. No fim do semestre, os melhores trabalhos na categoria de **melhor *bot*** receberão um prémio 🏆.

Funções a implementar

O objectivo desta tarefa é, no ficheiro `Tarefa4.hs`, definir a função

```
tatica :: Estado -> [(NumMinhoca,Jogada)]
```

cujo objectivo é calcular uma tática, isto é, uma sequência de jogadas para as várias minhocas disponíveis. Ou seja, uma tática será uma lista com exactamente 100 elementos, como por exemplo:

```
[ (0, Dispara Jetpack Norte), (1, Move Sul) ] ++ replicate 98 (2, Move Norte)
```

Para organizar melhor esta tarefa, a função `tatica` já vem pré-definida. Sugere-se que definam apenas a função:

```
jogadaTatica :: Ticks -> Estado -> (NumMinhoca, Jogada)
```

Como o nome indica, a função `jogadaTatica` efetua uma só jogada na tática. Para isso, recebe o tempo já decorrido (equivalente ao número de jogadas já efetuadas - 1) e o estado atual do jogo, e retorna um tuplo com uma minhoca e uma jogada a efetuar para essa minhoca. A título de exemplo, esta função pode ser definida como:

```
jogadaTatica t e = (0, Move Este)
```

Ou seja, a minhoca com índice 0 (se existir) irá sempre tentar mover-se para Este nas 100 jogadas disponíveis, independentemente do número da jogada e do estado atual.

Notem que a função `tatica` pré-definida calcula o `Estado` passado a cada `jogadaTatica` utilizando as vossas funções das Tarefas 2 e 3, de forma a simular a evolução do jogo, permitindo raciocinarem sobre cada jogada independentemente. Duas notas adicionais:

1. Uma limitação óbvia é que estamos a assumir que a função `jogadaTatica` decide cada jogada sem qualquer memória das jogadas efetuadas anteriormente. Para implementar táticas mais avançadas, considerem o template alternativo para a Tarefa 4 disponibilizado [aqui](#).
2. Se quiserem ainda mais controlo e em particular evitar a dependência das vossas tarefas anteriores, podem alterar diretamente o código da função `tatica`, e gerar uma sequência de jogadas sem utilizar o código fornecido.

Os grupos **não devem alterar** os nomes, tipos e assinaturas das funções previamente definidas, sob pena de não serem corretamente avaliados.

Consulte a [documentação online da Tarefa 4](https://haslab.github.io/Teaching/L11/2526/viewer.html), para mais detalhes. Pode jogar o jogo em <https://haslab.github.io/Teaching/L11/2526/viewer.html>. Em particular, repare que o visualizador online faz uma contabilização do dano total, por exemplo:

Pontuação (Dano)

Total (100) =  (30) +  (70)

Sistema de *Feedback*

Para obterem *feedback* sobre a Tarefa 4, devem declarar uma lista de testes no ficheiro de testes correspondente:

```
testesT4 :: [Estado]
```

De seguida, devem executar localmente no vosso projeto o comando:

```
cabal run t4-feedback
```

O comando irá compilar o código da tarefa juntamente com os testes, e deverá abrir no seu browser uma página web com uma listagem do feedback por cada teste. No caso da Tarefa 4, não existe uma noção de correção de cada teste. O sistema de feedback apenas vos permite observar como o vosso *bot* se irá comportar a partir de um determinado estado inicial. A avaliação automática desta tarefa será feita com estados definidos pelos docentes.

Tarefa 5 - Implementação da Interface gráfica em Gloss

O objetivo desta tarefa consiste em aproveitar todas as funcionalidades já elaboradas nas outras tarefas e construir uma aplicação gráfica que permita a um utilizador jogar o jogo. Para a interface gráfica, deverá usar a biblioteca [Gloss](#) estudada nas aulas práticas. Um template de como desenvolver uma aplicação gráfica em Gloss é disponibilizado na pasta `app` do vosso repositório Git. Para além de combinar as diferentes tarefas definidas anteriormente, o objetivo principal desta tarefa consiste em desenhar estados do jogo, o que será feito desenvolvendo uma função que converte um `Estado` no tipo `Picture` do Gloss. Consulte a [documentação online da Tarefa 5](#), para mais detalhes.

Notem que esta é uma tarefa aberta, que não terá avaliação automática. O grafismo e as funcionalidades disponibilizadas ficam à criatividade dos alunos. Em particular, incentiva-se que alterem as regras do jogo definidas nas tarefas anteriores¹ para melhorar o aspecto final e a jogabilidade do jogo. No fim do semestre, os melhores trabalhos na categoria de **melhor interface** receberão um prémio 🏆.

¹ Cuidado para não alterarem as regras do jogo para a Tarefa 4. Sugere-se que criem novas versões das funções das Tarefas 2 e 3 em novos módulos, diferentes dos utilizados na 1ª fase para evitar problemas.

Tarefa 6 - Extras

Esta tarefa destina-se a encorajar e recompensar a criatividade e o pensamento crítico dos alunos. Podem ser implementadas funcionalidades adicionais ou melhorias no jogo que não são explicitamente exigidas nas tarefas anteriores. Alguns exemplos de extras incluem:

- Adicionar a noção de equipas, de turnos ou de vitória do jogo. Considerar potencialmente modos de jogo alternativos, como dano máximo, last worm/team standing, etc.
- Adicionar novos tipos de armas com diferentes trajetórias e efeitos. Para inspiração, o jogo Worms original é bastante prolífico em tipos de armas.
- Adicionar novos tipos de terrenos com efeitos especiais, por exemplo lava que queima minhocas ou petróleo que prende minhocas.
- Adicionar sinergias novas entre projéteis. Por exemplo, considerar que quando dois disparos de Bazuca colidem estes explodem.
- Considerar projéteis com movimento contínuo em vez de discreto. Por exemplo, tal como no jogo Worms original, considerar que tiros de Bazuca têm um vetor de velocidade inicial definido aquando do disparo e descrevem uma parábola afetada por fatores como vento.
- Criar *powerups* que aparecem no mapa e podem ser apanhados pelas minhocas. Por exemplo, com mais vida ou munições de armas.
- Possibilitar guardar e carregar jogos, permitindo ao jogador retomar um jogo anteriormente guardado.
- Criar um sistema de progressão de jogo onde o jogador pode desbloquear novos mapas após vencer noutros.
- Implementar uma interface de configuração onde o jogador possa personalizar as regras do jogo, como o dano causado por armas ou se minhocas podem voar.
- Implementar uma interface gráfica de criação de mapas, onde o jogador pode desenhar o seu próprio mapa.
- Adicione outras funcionalidades gráficas, como animações, zoom, mudança de perspetiva (e.g. de 2D para 2.5D, e vice-versa), efeitos de partículas, vários estados de movimento ou de representação de minhocas consoante o seu estado, etc.

Mais exemplos de extras podem ser encontrados em [trabalhos de alunos de anos anteriores](#).

Os extras serão avaliados de acordo com a sua complexidade, criatividade, impacto na jogabilidade e qualidade da implementação. Alunos são incentivados a documentar claramente as suas ideias e justificações para as funcionalidades implementadas nos extras, detalhando os passos seguidos e as decisões tomadas.

Entrega e Avaliação

A data limite para conclusão de todas as tarefas desta primeira fase é **31 de Dezembro de 2025 às 23h59m59s (Portugal Continental)** e a respectiva avaliação terá um peso de 50% na nota final do projeto. A submissão será feita automaticamente através do Git: nesta data será feita uma cópia do repositório de cada grupo, sendo apenas considerados para avaliação os programas e demais artefactos que se encontrem no repositório nesse momento. O conteúdo dos repositórios será processado por ferramentas de detecção de plágio e, na eventualidade de serem detectadas cópias, estas serão consideradas fraude dando-se-lhes tratamento consequente.

Para além dos programas Haskell relativos às tarefas, será considerado parte integrante do projeto todo o material de suporte à sua realização armazenado no repositório Git do respectivo grupo (código, documentação, ficheiros de teste, etc.). A utilização das diferentes ferramentas abordadas na UC (como Haddock, Git, etc.) deve seguir as recomendações enunciadas nas respectivas sessões laboratoriais. A avaliação desta fase do projecto terá em linha de conta todo esse material, atribuindo-lhe os seguintes pesos relativos:

Componente	Peso
Avaliação automática da Tarefa 4	20%
Avaliação qualitativa da Tarefa 5	20%
Avaliação qualitativa da Tarefa 6	20%
Qualidade do código	20%
Relatório sob a forma de documentação Haddock do código	15%
Utilização do git e estrutura do repositório	5%

A avaliação automática será feita através de um conjunto de testes que não serão revelados aos grupos.

A avaliação qualitativa terá em conta a usabilidade e complexidade das interfaces gráficas desenvolvidas, juntamente com aspectos de qualidade de código (por exemplo, estrutura do código, elegância da solução implementada, etc.) e bom uso do Git como sistema de controle de versões.

Para além de utilizar o Haddock para gerar documentação de tipos e funções individuais, nesta 2ª fase deverão também escrever um relatório com recurso ao Haddock, sob a forma de comentários no código. O relatório deve:

- Ter uma organização que demonstra diferentes operadores sintáticos do Haddock.

- Descrever e justificar o racional da tática implementada para o *bot* (Tarefa 4).
- Descrever as funcionalidades implementadas (com particular ênfase nas não descritas no enunciado) na interface gráfica (Tarefas 5 e 6) e os desafios associados.